arXiv:2305.02412v2 [cs.CL] 7 May 2023

Plan, Eliminate, and Track — Language Models are Good Teachers for Embodied Agents.

Yue Wu [1] **So Yeon Min** [1] **Yonatan Bisk** [1] **Ruslan Salakhutdinov** [1] **Amos Azaria** [2] **Yuanzhi Li** [1 3]
Tom M. Mitchell [1] **Shrimai Prabhumoye** [4]

Abstract

Pre-trained large language models (LLMs) capture procedural knowledge about the world. Recent work has leveraged LLM's ability to generate abstract plans to simplify challenging control tasks, either by action scoring, or action modeling (fine-tuning). However, the transformer architecture inherits several constraints that make it difficult for the LLM to directly serve as the agent: e.g. limited input lengths, fine-tuning inefficiency, bias from pre-training, and incompatibility with non-text environments. To maintain compatibility with a low-level trainable actor, we propose to instead use the *knowledge* in LLMs to simplify the control problem, rather than solving it.

We propose the Plan, Eliminate, and Track (**PET**) framework. The Plan module translates a task description into a list of high-level sub-tasks. The Eliminate module masks out irrelevant objects and receptacles from the observation for the current sub-task. Finally, the Track module determines whether the agent has accomplished each sub-task. On the Alf-World instruction following benchmark, the **PET** framework leads to a significant 15% improvement over SOTA for generalization to human goal specifications.

1. Introduction

Humans can abstractly plan their everyday tasks without execution; for example, given the task "Make breakfast", we can roughly plan to first pick up a mug and make coffee, before grabbing eggs to scramble. Embodied agents, endowed with this capability will generalize more effectively by leveraging common-sense reasoning.

[1]Carnegie Mellon University [2]Ariel University [3]Microsoft Research [4]Nvidia Research. Correspondence to: Yue Wu <ywu5@andrew.cmu.edu>.

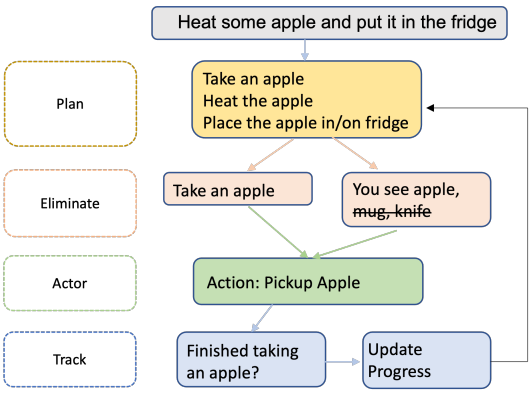

Figure 1. PET framework. Plan module uses LLM to generate a high-level plan. Eliminate Module uses a QA model to mask irrelevant objects in observation. Track module uses a QA model to track the completion of sub-tasks.

Recent work (Huang et al., 2022a;b; Ahn et al., 2022; Yao et al., 2020) has used LLMs (Bommasani et al., 2021) for abstract planning for embodied or gaming agents. These have shown incipient success in extracting procedural world knowledge from LLMs in linguistic form with posthoc alignment to executable actions in the environment. However, they treat LLMs as the actor, and focus on adapting LLM outputs to executable actions either through fine-tuning (Micheli & Fleuret, 2021) or constraints (Ahn et al., 2022). Using LLM as the actor works for pure-text environments with limited interactions (Huang et al., 2022b; Ahn et al., 2022) (just consisting of "picking/placing" objects), but limits generalization to other modalities. In addition, the scenarios considered have been largely simplified from the real world. Ahn et al. (2022) provides all available objects and possible interactions at the start and limits tasks to the set of provided objects/interactions. Huang et al. (2022b) limits the environment to objects on a single table.

On the other hand, to successfully "cut some lettuce" in a real-world room, one has to "find a knife", which can be non-trivial since there can be multiple drawers or cabinets (Chaplot et al., 2020; Min et al., 2021; Blukis et al., 2021). A more realistic scenario leads to a

diverse, complicated set of tasks or large and changing action space. Furthermore, the text description of the observation increases as a function of the number of receptacles and objects the agent sees. Combined with growing roll-outs, the state becomes too verbose to fit into any LLM.

In this work, we explore alternative mechanisms to leverage the prior knowledge encoded in LLMs without impacting the trainable nature of the actor. We propose a 3-step framework (Figure 1): Plan, Eliminate, and Track (PET). **Plan** module simplifies complex tasks by breaking them down into sub-tasks. It uses a pre-trained LLM to generate a list of sub-tasks for an input task description employing example prompts from the training set similar to Huang et al. (2022a); Ahn et al. (2022). The **Eliminate** module addresses the challenge of long observations. It uses a zero-shot QA language model to score and mask objects and receptacles that are irrelevant to the current sub-task. The **Track** module uses a zero-shot QA language model to determine if the current sub-task is complete and moves to the next sub-task. Finally, the **Action Attention** agent uses a transformer-based architecture to accommodate for long roll-out and variable length action space. The agent observes the masked observation and takes an action conditioned on the current sub-task.

We focus on instruction following in indoor households on the AlfWorld (Shridhar et al., 2020b) interactive text environment benchmark. Our experiments and analysis demonstrate that LLMs not only remove 40% of task-irrelevant objects in observation through common-sense QA, but also generate high-level sub-tasks with 99% accuracy. In addition, multiple LLMs may be used in coordination with each other to assist the agent from different aspects.

Our contributions are as follows:

1. **PET**: A novel framework for leveraging pre-trained LLMs with embodied agents; our work shows that each of P, E, T serves a complementary role and should be simultaneously addressed to tackle control tasks.
2. An Action Attention agent that handles the changing action space for text environments.
3. A 15% improvement over SOTA for generalization to human goals via sub-task planning and tracking.

2. Related Work

Language Conditioned Policies A considerable portion of prior work studies imitation learning (Tellex et al., 2011; Mei et al., 2016; Nair et al., 2022; Stepputtis et al., 2020; Jang et al., 2022; Shridhar et al., 2022; Sharma et al., 2021) or reinforcement learning (Misra et al., 2017; Jiang et al., 2019; Cideron et al., 2020; Goyal et al., 2021; Nair et al., 2022; Akakzia et al., 2020) policies conditioned on natural language instruction or goal (MacMahon et al., 2006; Kollar et al., 2010). While some prior research has used pre-trained language embeddings to improve generalization to new instructions (Nair et al., 2022), they lack domain knowledge that is captured in LLMs. Our PET framework enables planning, progress tracking, and observation filtering through the use of LLMs, and is designed to be compatible with any language conditional policies above.

LLMs for Control LLMs have recently achieved success in high-level planning. Huang et al. (2022a) shows that pre-trained LLMs can generate plausible plans for day-to-day tasks, but the generated sub-tasks cannot be directly executed in an end-to-end control environment. Ahn et al. (2022) solves the executability issue by training an action scoring model to re-weigh LLM action choices and demonstrates success on a robot. However, LLM scores work for simple environments with actions limited to pick/place (Ahn et al., 2022), but fails with environments with more objects and diverse actions (Shridhar et al., 2020b). Song et al. (2022) uses GPT3 to generate step-by-step low-level commands, which are then executed by respective control policies. the work improves Ahn et al. (2022) with more action diversity and on-the-fly re-plan. In addition, all the above LLMs require few-shot demonstrations of up to 17 examples, making the length of the prompt infeasible for AlfWorld. Micheli & Fleuret (2021) fine-tuned a GPT2-medium model on expert trajectories in AlfWorld and demonstrated impressive evaluation results. However, LM fine-tuning requires a fully text-based environment, consistent expert trajectories, and a fully text-based action space. Such requirements greatly limit the generalization to other domains, and even to other forms of task specification. We show that our PET framework achieves better generalization to human goal specifications which the agents were not trained on.

Hierarchical Planning with Natural Language Due to the structured nature of natural language, Andreas et al. (2017) explored associating each task description to a modular sub-policy. Later works extend the above approach by using a single conditional policy (Mei et al., 2016), or by matching sub-tasks to templates (Oh et al., 2017). Recent works have shown that LLMs are proficient high-level planners (Huang et al., 2022a; Ahn et al., 2022; Lin et al., 2022), and therefore motivates us to revisit the idea of hierarchical task plan-

ning with progress tracking. To our knowledge, PET is the first work combining a zero-shot subtask-level LLM planner and zero-shot LLM progress tracker with a low-level conditional sub-task policy.

Text Games Text-based games are complex, interactive simulations where the game state and action space are in natural lanugage. They are fertile ground for language-focused machine learning research. In addition to language understanding, successful play requires skills like memory and planning, exploration (trial and error), and common sense. The AlfWorld (Shridhar et al., 2020b) simulator extends a common text-based game simulator, TextWorld Côté et al. (2018a), to create text-based analogs of each ALFRED scene.

Agents for Large Action Space He et al. (2015) learns representation for state and actions with two different models and computes the Q function as the inner product of the representations. While this could generalize to large action space, they only considered a small number of actions.

Fulda et al. (2017); Ahn et al. (2022) explore action elimination in the setting of affordances. Zahavy et al. (2018) trains a model to eliminate invalid actions on Zork from external environment signals. However, the functionality depends on the existence of external elimination signal.

3. Plan, Eliminate, and Track

In this section, we explain our 3-step framework: Plan, Eliminate, and Track (PET). In **Plan** module ($\mathcal{M}_\mathbf{P}$), a pre-trained LLM generates a list of sub-tasks for an input task description using samples from the training set as in-context examples. The **Eliminate** module ($\mathcal{M}_\mathbf{E}$) uses a zero-shot QA language model to score and mask objects and receptacles that are irrelevant to the current sub-task. The **Track** module ($\mathcal{M}_\mathbf{T}$) uses a zero-shot QA language model to determine if the current sub-task is complete and moves to the next sub-task. Note that Plan is a generative task and Eliminate and Track are classification tasks.

We also implement an attention-based **agent** (Action Attention), which scores each permissible action and is trained on imitation learning on the expert. The agent observes the masked observation and takes an action conditioned on the current sub-task.

Problem Setting We define the task description as $\mathcal{T}$, the observation string at time step t as $\mathcal{O}^t$, and the list of permissible actions $\{a_i^t | a_i^t \text{ can be executed}\}$ as A^t. For each observation string $\mathcal{O}^t$, we define the receptacles and objects within the observation as r_i^t and o_i^t respectively. The classification between receptacles and objects is defined by the environment (Shridhar et al., 2020b). For a task $\mathcal{T}$, we assume there exists a list of sub-tasks $\mathcal{S}_\mathcal{T} = \{s_1, \dots s_k\}$ that solves $\mathcal{T}$.

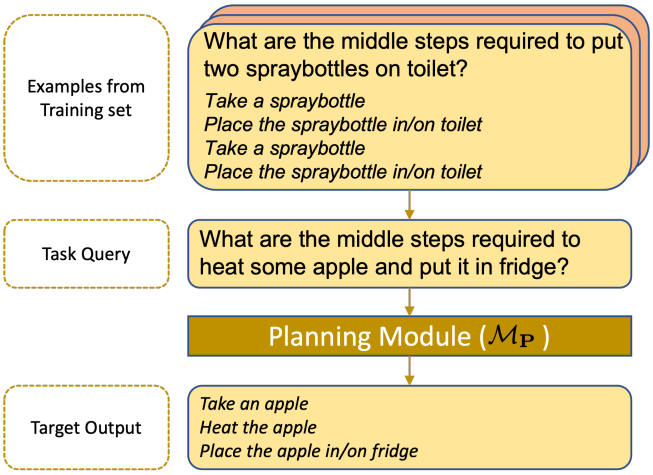

Figure 2. Plan Module (Sub-task Generation). 5 full examples are chosen from the training set based on RoBERTa embedding similarity with the task query description. Then the examples are concatenated with the task query to get the prompt. Finally, we prompt the LLM to generate the desired sub-tasks.

3.1. Plan

Tasks in the real world are often complex and need more than one step to be completed. Motivated by the ability of humans to plan high-level sub-tasks given a complex task, we design the **Plan** module ($\mathcal{M}_\mathbf{P}$) to generate a list of high-level sub-tasks for a task description $\mathcal{T}$.

Inspired by the contextual prompting techniques for planning with LLMs (Huang et al., 2022a), we use an LLM as our plan module $\mathcal{M}_\mathbf{P}$. For a given task description $\mathcal{T}$, we compose the query question $\mathcal{Q}_\mathcal{T}$ as "What are the middle steps required to $\mathcal{T}$?", and require $\mathcal{M}_\mathbf{P}$ to generate a list sub-tasks $\mathcal{S}_\mathcal{T} = \{s_1, \dots s_k\}$.

Specifically, we select the top 5 example tasks $\mathcal{T}^E$ from the training set based on RoBERTa (Liu et al., 2019) embedding similarity with the query task $\mathcal{T}$. We then concatenate the example tasks with example sub-tasks in a query-answer format to build the prompt $\mathcal{P}_\mathcal{T}$ for $\mathcal{M}_\mathbf{P}$ (Fig. 2):

$$\mathcal{P}_\mathcal{T} = \texttt{concat}(\mathcal{Q}_{\mathcal{T}_1^E}, \mathcal{S}_{\mathcal{T}_1^E}, \dots, \mathcal{Q}_{\mathcal{T}_5^E}, \mathcal{S}_{\mathcal{T}_5^E}, \mathcal{Q}_\mathcal{T})$$

An illustration of our prompt format is shown in Figure 2, where $\mathcal{T}$ ="heat some apple and put it in fridge", and $\mathcal{Q}_{\mathcal{T}_1^E}$ ="What are the middle steps required to put two spraybottles on toilet", $\mathcal{S}_{\mathcal{T}_1^E}$ ="take a spraybottle,

place the spraybottle in/on toilet, take a spraybottle, place the spraybottle in/on toilet". The expected list of sub-tasks to achieve this task $\mathcal{T}$ is s_1 ='take an apple', s_2 ='heat the apple', and s_3 ='place the apple in/on fridge'

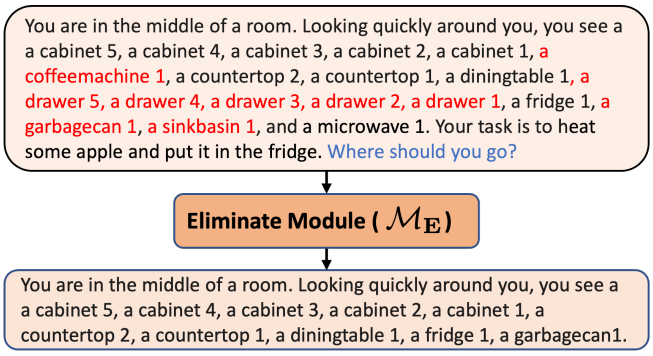

Figure 3. Eliminate Module (Receptacle Masking). We use a pre-trained QA model to filter irrelevant receptacles/objects in the observation of each scene. As we can see, the original observation is too long and the receptacles shown in red are not relevant for task completion. These receptacles are filtered by the QA model making the observation shorter.

3.2. Eliminate

Typical Alfworld scenes can start with around 15 receptacles, each containing up to 15 objects. In some close-to-worst cases, there can be around 30 open-able receptacles (e.g. a kitchen with many cabinets and drawers), and it easily takes an agent with no prior knowledge more than 50 steps for the agent to find the desired object (repeating the process of visiting each receptacle, opening it, closing it). We observe that many receptacles and objects are irrelevant to specific tasks during both training and evaluation, and can be easily filtered with common-sense knowledge about the tasks. For example, in Fig. 3 the task is to heat some apple. By removing the irrelevant receptacles like the coffeemachine, garbagecan, or objects like knife, we could significantly shorten our observation. We therefore propose to leverage commonsense knowledge captured by large pre-trained QA models to design our Eliminate module $\mathcal{M}_{\mathbf{E}}$ to mask out irrelevant receptacles and objects.

For task $\mathcal{T}$, we create prompts in the format $\mathcal{P}_r$ ="Your task is to: $\mathcal{T}$. Where should you go to?" for receptacles and $\mathcal{P}_o$ ="Your task is to: $\mathcal{T}$. Which objects will be relevant?" for objects. Using the pre-trained QA model $\mathcal{M}_{\mathbf{E}}$ in a zero-shot manner, we compute score $\mu_{o_i} = \mathcal{M}_{\mathbf{E}}(\mathcal{P}_o, o_i)$ for each object o_i and $\mu_{r_i} = \mathcal{M}_{\mathbf{E}}(\mathcal{P}_o, r_i)$ for each receptacle r_j in observation at every step. μ represents the belief score of whether the common-sense QA model believes the object/receptacle is relevant to $\mathcal{T}$. We then remove o_i from observation if $\mu_{o_i} < \tau_o$, and remove r_i if $\mu_{r_i} < \tau_r$. Threshold τ_o, τ_r are hyper-parameters.

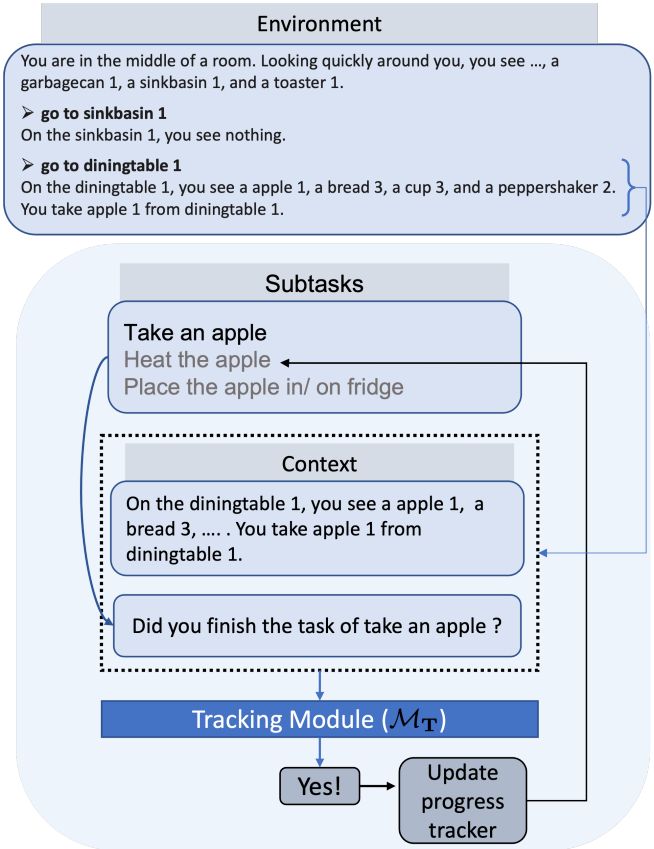

Figure 4. Track Module (Progress Tracking). At every step, we take the last 3 steps of roll-out as context and append a query (about whether the current sub-task is completed) to get the prompt. A pre-trained QA model generates a Yes/No answer to the prompt. For the answer "Yes", we update the tracker to the next sub-task.

3.3. Track

For the agent to utilize the high-level plan, it first needs to know which sub-task to execute. A human actor typically starts from the first item and check-off the tasks one by one until completion. Therefore, similar to Section 3.2, we use a pre-trained QA model to design the Track module $\mathcal{M}_{\mathbf{T}}$ to perform zero-shot sub-task completion detection.[1]

Specifically, as illustrated in Figure 4, for sub-task list $\mathcal{S}_{\mathcal{T}} = \{s_1, \ldots s_k\}$, we keep track of a progress tracker p (initialized at 1) that indicates the sub-task the agent is currently working on (s_p). We then compose the context as the last d steps of the agent observation

[1]Note that the current system design does not allow re-visiting finished sub-tasks, so the agent has no means to recover if it undoes its previous sub-task at test time.

for the current sub-task and the question as "Did you finish the task of s_p?". For efficiency, we set $d := \min(d+1, 3)$ at each step. Note that d is reset to 1 whenever the progress tracker updates. Hence, the template $\mathcal{P}_a = \texttt{concat}(\mathcal{O}^{t-d}, \ldots, \mathcal{O}^{t-1}$, "Did you finish the task of s_p?"). We feed $\mathcal{P}_a$ to a pre-trained zero-shot QA model $\mathcal{M}_{\mathbf{T}}$ and compute the probability of tokens 'Yes' and 'No' as follows: $p_{\mathcal{M}_{\mathbf{T}}}(\text{"}Yes\text{"}|\mathcal{P}_a)$ and $p_{\mathcal{M}_{\mathbf{T}}}(\text{"}No\text{"}|\mathcal{P}_a)$. If $p_{\mathcal{M}_{\mathbf{T}}}(\text{"}Yes\text{"}|\mathcal{P}_a) > p_{\mathcal{M}_{\mathbf{T}}}(\text{"}No\text{"}|\mathcal{P}_a)$ then we increment the tracker p to track the next sub-task.

If the tracking ends prematurely, meaning that $p > len(\mathcal{S}_{\mathcal{T}})$ but the environment has not returned "done", we fall back to conditioning with $\mathcal{T}$. We study the rate of pre-mature ends in Section 4.4 in terms of precision and recall.

3.4. Agent

Since the number of permissible actions can vary a lot by the environment, the agent needs to handle arbitrary dimensions of action space. While Shridhar et al. (2020b) addresses this challenge by generating actions token-by-token, such a generation process leads to degenerate performance even on the training set.

We draw inspiration from the field of text summarization, where models are built to handle variable input lengths. See et al. (2017) generates a summary through an attention-like "pointing" mechanism that extracts the output word by word. Similarly, an attention-like "pointing" model could be used to select an action from the list of permissible actions.

Action Attention We are interested in learning a policy π that outputs the optimal action among permissible actions. We eschew the long rollout/ large action space problems by (1) representing observations by averaging over history, and (2) individually encoding actions (Fig 5). In our proposed action attention framework, we first represent historical observations H^t as the average of embeddings of all individual observations through history (Eq. 1), and H^A as the list of embeddings of all the current permissible actions (Eq. 2). Then, in Eq. 3, we compute the query Q using a transformer with a "query" head ($\mathcal{M}_{\mathcal{Q}}$) on task embedding ($H^t$), the current observation embedding ($\mathcal{O}^t$), and the list of action embeddings (H^A). In Eq. 4 we compute the key K_i for each action a_i using the same transformer with a "key" head ($\mathcal{M}_{\mathcal{K}}$) on task embedding ($H^t$), the current observation embedding ($\mathcal{O}^t$), and embedding of action (a_i).

Finally, we compute the dot-product of the query and keys as action scores for the policy π (Eq. 5).

$$H^t = \text{avg}_{j\in[1,t-1]}\text{Embed}(\mathcal{O}^j) \tag{1}$$

$$H^A = \left[\text{Embed}(a_1^t), ..., \text{Embed}(a_n^t)\right] \tag{2}$$

$$Q = \mathcal{M}_{\mathcal{Q}}\left(\text{Embed}(\mathcal{T}), H^t, \text{Embed}(\mathcal{O}^t), H^A\right) \tag{3}$$

$$K_i = \mathcal{M}_{\mathcal{K}}\left(\text{Embed}(\mathcal{T}), H^t, \text{Embed}(\mathcal{O}^t), \text{Embed}(a_i^t)\right) \tag{4}$$

$$\pi = \text{softmax}\left([Q \cdot K_i | i \in \text{all permissible actions}]\right) \tag{5}$$

4. Experiments and Results

We present our experiments as follows. First, we explain the environment setup and baselines for our experiments. Then we compare PET to the baselines on different splits of the environment. Finally, we conduct ablation studies and analyze the PET framework part by part. We show that PET generalizes better to human goal specification under efficient behavior cloning training.

4.1. Experimental Details

AlfWorld Environment ALFWorld (Shridhar et al., 2020b) is a set of TextWorld environments (Côté et al., 2018b) that are parallels of the ALFRED embodied dataset (Shridhar et al., 2020a). ALFWorld includes 6 task types that each require solving multiple compositional sub-goals. There are 3553 training task instances ({tasktype, object, receptacle, room}), 140 in-distribution evaluation task instances (seen split - tasks themselves are novel but take place in rooms seen during training) and 134 out-of-distribution evaluation task instances (unseen split - tasks take place in novels rooms). An example of the task could be: "Rinse the egg to put it in the microwave." Each training instance in AlfWorld comes with an expert, from which we collected our training demonstration.

Human Goal Specification The crowd-sourced human goal specifications for evaluation contain 66 unseen verbs and 189 unseen nouns (Shridhar et al., 2020b). In comparison, the template goals use only 12 ways of goal specification. In addition, the sentence structure for human goal specification is more diverse compared to the template goals. Therefore, human goal experiments are good for testing the generalization of models to out-of-distribution scenarios.

Pre-trained LMs. For the **Plan** module (sub-task generation), we experimented with the open-source GPT-Neo-2.7B (Black et al., 2021), and an industry-scale LLM with 530B parameters (Smith et al., 2022).

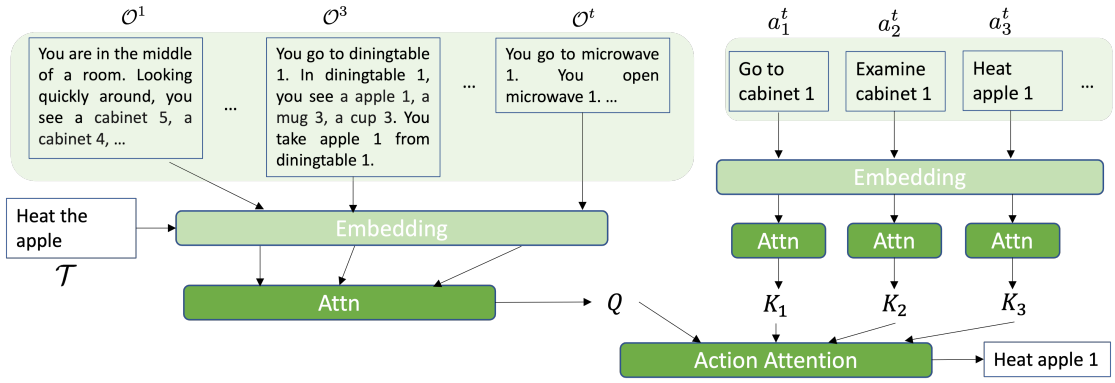

Figure 5. Agent (Action Attention). Action Attention block is a transformer-based framework that computes a key K_i for each permissible action and output action scores as dot-product between key and query Q from the observations.

	Template Goal Specification		Human Goal Specification	
Model	seen	unseen	seen	unseen
BUTLER + DAgger* (Shridhar et al., 2020b)	40	35	8	3
BUTLER + BC (Shridhar et al., 2020b)	10	9	-	-
GPT (Micheli & Fleuret, 2021)	**91**	**95**	42	57
PET + Action Attention (Ours)	70	67.5	**52.5**	**60**

Table 1. Comparison of different models in terms of completion rate per evaluation split (seen and unseen), with and without human annotated goals. PET under-performs GPT on Template goal specifications but generalizes better to human goal specifications. * We include the performance of BUTLER with DAgger for completeness. All other rows are trained without interaction with the environment, MLE for GPT and behavior cloning for BUTLER+BC and PET.

For the **Eliminate** module (receptacle/object masking), we choose Macaw-11b (Tafjord & Clark, 2021), which is reported to have common sense QA performance on par with GPT3 (Brown et al., 2020) while being orders of magnitudes smaller. We use a decision threshold of 0.4 for Macaw score below which the objects are masked out. For the **Track** module (progress tracking), we use the same Macaw-11b model as the Eliminate module answer to Yes/No questions.

Actor Model Design. Our **Action Attention** agent ($\mathcal{M}_\mathcal{Q}$ and $\mathcal{M}_\mathcal{K}$) is a 12-layer transformer with 12 heads and hidden dimension 384. The last layer is then fed into two linear heads to generate K and Q. For embedding of actions and observations, we use pre-trained RoBERTa-large (Liu et al., 2019) with embedding dimension 1024. For sub-task generation, we use ground-truth sub-tasks for training, and generated sub-tasks from Plan module for evaluation.

Experimental Setup. Unlike the original benchmark (Shridhar et al., 2020b), we experiment with models trained with behavior cloning. Although Shridhar et al. (2020b) observe that models benefit greatly from DAgger training, DAgger assumes an expert that is well-defined at all possible states, which is inefficient and impractical. In our experiments, training is 100x slower with DAgger compared to behavior cloning (3 weeks for DAgger v.s. 6 hours for Behavior Cloning). In addition, we demonstrate that our models surpass the DAgger training performance of the BUTLER (Shridhar et al., 2020b) agents trained with DAgger, even when our agent does not have the option to interact with the environment.

Baselines. Our first baseline is the BUTLER::BRAIN (**BUTLER**) agent (Shridhar et al., 2020b), which consists of an encoder, an aggregator, and a decoder. At each time step t, the encoder takes initial observation s^0, current observation s^t, and task string s_{task} and generates representation r^t. The recurrent aggregator combines r^t with the last recurrent state h^{t-1} to produce h^t, which is then decoded into a string a^t representing action. In addition, the BUTLER agent uses beam search to get out of stuck conditions in the event of a failed action. Our second baseline **GPT** (Micheli & Fleuret, 2021) is a fine-tuned GPT2-medium on 3553 demonstrations from the AlfWorld training set. Specifically, the GPT is fined-tuned to generate each action step word-by-word to mimic the rule-based expert using the standard maximum likelihood loss.

4.2. Overall Results on Template and Human Goals

We compare the performance of action attention assisted by PET with BUTLER (Shridhar et al., 2020b) and fine-tuned GPT (Micheli & Fleuret, 2021) in Table 1. For human goal specifications, PET outperforms SOTA (GPT) by 25% on seen and 5% on the unseen split.

Although PET under-performs GPT on Template goal specifications, GPT requires fine-tuning on fully text-based expert trajectory and thus loses adaptability to different environment settings. Qualitatively, on human goal specification tasks, where the goal specifications are out-of-distribution, GPT often gets stuck repeating the same action after producing a single wrong move. On the other hand, since the Plan module of PET is not trained on the task, it generalizes to the variations for human goal specifications as shown in Section 4.5. Quantitatively, GPT suffers from a relative 50% performance drop transferring from template to human-goal specifications, whereas PET incurs only a 15 $\sim$ 25% drop.

The setting closest to PET is BUTLER with behavior cloning (BUTLER + BC). Since BUTLER + BC performs poorly, we also include DAgger training results. Nevertheless, action attention assisted by PET outperforms BUTLER with DAgger by more than 2x while being much more efficient. (Section 4.1)

4.3. Ablations for Plan, Eliminate, and Track

In Table 3, we analyze the contribution of each PET module by sequentially adding each component to the action attention agent on 140 training trajectories sampled from the training set. The data set size is chosen to match the size of the seen validation set, for an efficient and sparse setting. Note that we treat Plan and Track as a single module for this ablation since they cannot work separately.

Adding Plan and Track greatly improves the completion rate relatively by 60%, which provides evidence to our hypothesis that solving some embodied tasks step-by-step reduces the complexity. We observe a relatively insignificant 3% improvement in absolute performance when adding Eliminate without sub-task tracking. On the other hand, when applying Eliminate to sub-tasks with Plan and Track, we observe more than 60% relative improvement over just Plan and Track alone. We, therefore, deduce that Plan and Track boost the performance of Eliminate during evaluation, since it is easier to remove irrelevant objects when the objective is more focused on sub-tasks.

4.4. Automated Analysis of PET modules

Plan Module We experiment with different LLMs such as GPT2-XL (Radford et al., 2019), GPT-Neo-2.7B (Black et al., 2021), and the 530B parameter MT-NLG (Smith et al., 2022) models. Table 2 reports the generation accuracy and the RoBERTa (Liu et al., 2019) embedding cosine similarity against ground-truth sub-tasks. We observe that all LLMs achieve high accuracy on template goal specifications, where there is no variation in sentence structures. For human goal specification, MT-NLG generates subtasks similar to ground truth in terms of embedding similarity, while the other smaller models perform significantly worse.

Eliminate module We evaluate the zero-shot receptacle/object masking performance of Macaw on the three splits of AlfWorld. In Fig 6, we illustrate the AUC curve of the relevance score that the model assigns to the objects v.s. objects that the rule-based expert interacted with when completing each task. Since the Macaw QA model is queried in a zero-shot manner, it demonstrates consistent masking performance on all three splits of the environment, even on the unseen split. In addition, we note that object receptacle accuracy is generally lower than object accuracy because of the counter-intuitive spawning locations described in Section 4.5. In our experiments, a decision threshold of 0.4 has a recall of 0.91 and reduces the number of objects in observation by 40% on average.

Track module Since sub-task alignment information is not provided by the environment, we explore an alternative performance metric for the detection of the event of completion. Ideally, a sub-task tracker should record the last sub-task as "finished" if and only if the environment is "fully solved" by the expert. As an agreement measure, we report a precision of 0.99 and a recall of 0.78 for Macaw-11B and a precision of 0.96 and a recall of 0.96 for Macaw-large. The larger model (Macaw-11b) is more precise but misses more detection, therefore limiting the theoretical performance to 78%. The smaller model is much less accurate according to human evaluation but does not limit the overall model performance in theory. In our experiments, we find that both models produce similar overall results, which may suggest that the overall results could be improved with LLMs doing better on both precision and recall.

4.5. Qualitative Analysis

Plan Module We show two types of failure examples for sub-task generation in Table 4. The first type of error is caused by generating synonyms of the ground truth, and the second type of error is caused by inaccu-

	Template Goals		Human Goals	
LLM	seen	unseen	seen	unseen
GPT-2 (Radford et al., 2019)	94.29 (0.97)	87.31 (0.94)	10.07 (0.62)	7.98 (0.58)
GPT-Neo-2.7B (Black et al., 2021)	**99.29 (1.00)**	96.27 (0.98)	4.70 (0.82)	9.16 (0.80)
MT-NLG (Smith et al., 2022)	98.57 (0.99)	**100 (1.00)**	**40.04 (0.94)**	**49.3 (0.94)**

Table 2. Evaluation of different LLMs for **Plan** module in terms of accuracy and RoBERTa embedding cosine similarity (in brackets) against ground-truth sub-tasks, per evaluation split (seen and unseen), with and without human annotated goals. The MT-NLG with 530B parameters achieves the overall best performance on all dataset splits and greatly exceeds the performance of smaller models on hard tasks with human goal specification. In addition, MT-NLG generates sub-tasks with almost perfect embedding similarity for all tasks.

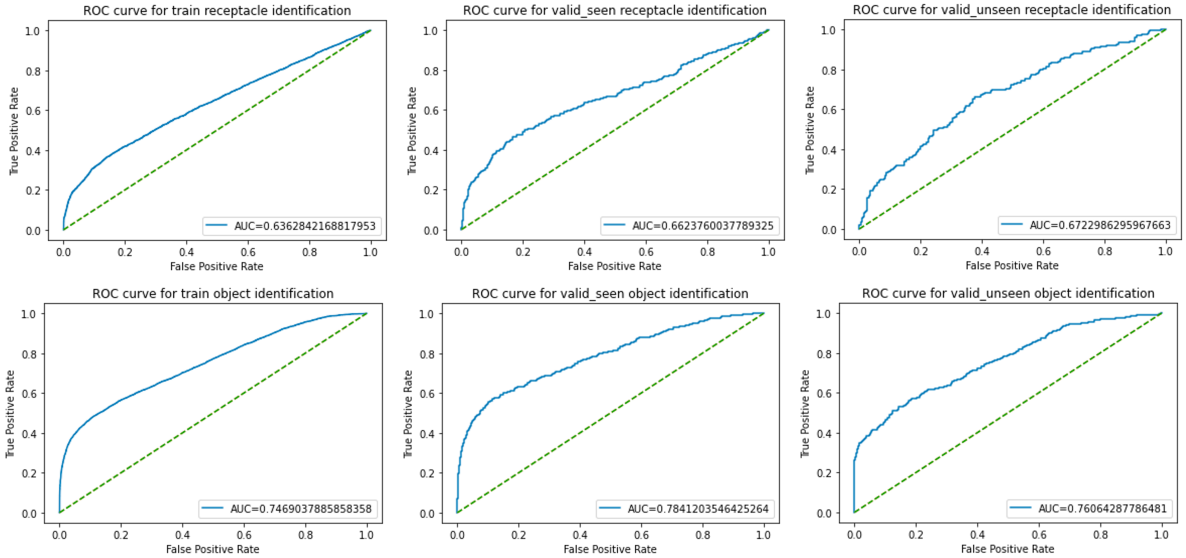

Figure 6. Plot of AUC scores of zero-shot relevance identification across all tasks in the Alfworld-Thor environment, with the Macaw-11b model. The ground truth is obtained as receptacles/objects accessed by the rule-based expert. **Top:** Receptacle relevance identification. **Bottom:** Object relevance identification. The QA model achieves an average AUC-ROC score of 65 for receptacles and 76 on objects.

Model Ablations	seen	unseen
Action Attention	25	9
Action Attention + Eliminate	25	11
Action Attention + Plan & Track	35	15
Action Attention + PET	52.5	27.5

Table 3. Comparison of different ablations of PET trained on a sampled set of 140 demonstrations from the training set, in terms of completion rate per evaluation split (seen and unseen). Applying Eliminate module alone has an insignificant effect on overall performance compared to Plan & Track. However, applying Eliminate module on sub-tasks together with Plan & Track results in a much more significant performance improvement.

racies in the human goal specifications. Note that our Action Attention framework uses RoBERTa (Liu et al., 2019) embedding for sub-tasks, known to be robust to synonym variations.

Eliminate Module We observe that the main source of elimination error occurs when the module incorrectly masks a receptacle that contains the object of interest so the agent fails to find such receptacles. This is often because some objects in the AI2Thor simulator do not spawn according to common sense. As noted in the documentation of the environment[2], objects like Apple or Egg has a chance of spawning in unexpected receptacles like GarbageCan, or TVStand. However, such generations in AI2Thor are unlikely in real deployment; thus, the "mistakes" of our Eliminate module are reasonable.

Track Module Experimentally, we find that sub-task planning/tracking is particularly helpful for tasks that require counting procedures. As shown in Table **??**, PET breaks the task of "Place two soapbar in cabinet" into two repeating set of sub-tasks: "take soapbar→place soapbar in/on cabinet". Sub-task planning and tracking, therefore, simplify the hard problem of counting.

[2] ai2thor.allenai.org/ithor/documentation/objects/object-types/

	Human Goal Specification Examples
Task	Chill a cup and place it in the cabinet.
GT	*cool* the mug→*place* the mug *in/on* coffeemachine
Gen	*chill* the mug→*return* the mug *to* coffeemachine
Task	Take the pencil from the desk, put it on the other side of the desk
GT	take a pencil→place the pencil in/on shelf
Gen	pick up the white pencil on the desk→put the white pencil on another spot on the desk

Table 4. Failure examples from the Plan module on human goal specifications (Task), ground-truth (GT) v.s. generated (Gen). In the first example, generated plan differs from the ground truth but the meaning agrees. In the second example, the generated plan largely differs from the ground truth due to the mistake in human goal specification — "another side on the desk" instead of "shelf".

5. Conclusion, Limitations, and Future Work

In this work, we propose the Plan, Eliminate, and Track (PET) framework that uses pre-trained LLMs to assist an embodied agent in three steps. Our PET framework requires no fine-tuning and is designed to be compatible with any goal-conditional embodied agents.

In our experiments, we combine PET with a novel Action Attention agent that handles the dynamic action space in AlfWorld. Our Action Attention agent greatly outperforms the BUTLER baseline. In addition, since the PET framework is not trained to fit the training set tasks, it demonstrates better generalization to unseen human goal specification tasks. Finally, our ablation studies show the Plan and Track modules together improve the performance of Eliminate module to achieve the best performance.

Our results show that LLMs can be a good source of common sense and procedural knowledge for embodied agents, and multiple LLMs may be used in coordination with each other to further improve effectiveness.

One of the major limitations of our current system design is that the Track module (progress tracker) does not re-visit finished sub-tasks. If for example, the agent is executing sub-tasks [picked up a pan, put the pan on countertop], and it picked up a pan but put it in the fridge (undo pickup action). Since the progress tracker does not take into consideration previous progress being undone, the system may break in this situation. Future work can focus on adding sub-task-level dynamic replanning to address this limitation or explore other ways in which LLMs can assist the learning of the policy (i.e., reading an instruction manual about the environment).

References

Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Fu, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Ho, D., Hsu, J., Ibarz, J., Ichter, B., Irpan, A., Jang, E., Ruano, R. J., Jeffrey, K., Jesmonth, S., Joshi, N. J., Julian, R., Kalashnikov, D., Kuang, Y., Lee, K.-H., Levine, S., Lu, Y., Luu, L., Parada, C., Pastor, P., Quiambao, J., Rao, K., Rettinghouse, J., Reyes, D., Sermanet, P., Sievers, N., Tan, C., Toshev, A., Vanhoucke, V., Xia, F., Xiao, T., Xu, P., Xu, S., Yan, M., and Zeng, A. Do as i can, not as i say: Grounding language in robotic affordances, 2022. URL https://arxiv.org/abs/2204.01691.

Akakzia, A., Colas, C., Oudeyer, P.-Y., Chetouani, M., and Sigaud, O. Grounding language to autonomously-acquired skills via goal generation. *arXiv preprint arXiv:2006.07185*, 2020.

Andreas, J., Klein, D., and Levine, S. Modular multi-task reinforcement learning with policy sketches. In *International Conference on Machine Learning*, pp. 166–175. PMLR, 2017.

Black, S., Gao, L., Wang, P., Leahy, C., and Biderman, S. GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow, March 2021. URL https://doi.org/10.5281/zenodo.5297715. If you use this software, please cite it using these metadata.

Blukis, V., Paxton, C., Fox, D., Garg, A., and Artzi, Y. A persistent spatial semantic representation for high-level natural language instruction execution, 2021. URL https://arxiv.org/abs/2107.05612.

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., Brynjolfsson, E., Buch, S., Card, D., Castellon, R., Chatterji, N., Chen, A., Creel, K., Davis, J. Q., Demszky, D., Donahue, C., Doumbouya, M., Durmus, E., Ermon, S., Etchemendy, J., Ethayarajh, K., Fei-Fei, L., Finn, C., Gale, T., Gillespie, L., Goel, K., Goodman, N., Grossman, S., Guha, N., Hashimoto, T., Henderson, P., Hewitt, J., Ho, D. E., Hong, J., Hsu, K., Huang, J., Icard, T., Jain, S., Jurafsky, D., Kalluri, P., Karamcheti, S., Keeling, G., Khani, F., Khattab, O., Koh, P. W., Krass, M., Krishna, R., Kuditipudi, R., Kumar, A., Ladhak, F., Lee, M., Lee, T., Leskovec, J., Levent, I., Li, X. L., Li, X., Ma,

T., Malik, A., Manning, C. D., Mirchandani, S., Mitchell, E., Munyikwa, Z., Nair, S., Narayan, A., Narayanan, D., Newman, B., Nie, A., Niebles, J. C., Nilforoshan, H., Nyarko, J., Ogut, G., Orr, L., Papadimitriou, I., Park, J. S., Piech, C., Portelance, E., Potts, C., Raghunathan, A., Reich, R., Ren, H., Rong, F., Roohani, Y., Ruiz, C., Ryan, J., Ré, C., Sadigh, D., Sagawa, S., Santhanam, K., Shih, A., Srinivasan, K., Tamkin, A., Taori, R., Thomas, A. W., Tramèr, F., Wang, R. E., Wang, W., Wu, B., Wu, J., Wu, Y., Xie, S. M., Yasunaga, M., You, J., Zaharia, M., Zhang, M., Zhang, T., Zhang, X., Zhang, Y., Zheng, L., Zhou, K., and Liang, P. On the opportunities and risks of foundation models, 2021. URL https://arxiv.org/abs/2108.07258.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Chaplot, D. S., Gandhi, D., Gupta, A., and Salakhutdinov, R. Object goal navigation using goal-oriented semantic exploration, 2020. URL https://arxiv.org/abs/2007.00643.

Cideron, G., Seurin, M., Strub, F., and Pietquin, O. Higher: Improving instruction following with hindsight generation for experience replay. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 225–232. IEEE, 2020.

Côté, M.-A., Kádár, A., Yuan, X., Kybartas, B., Barnes, T., Fine, E., Moore, J., Hausknecht, M., Asri, L. E., Adada, M., et al. Textworld: A learning environment for text-based games. In *Workshop on Computer Games*, pp. 41–75. Springer, 2018a.

Côté, M.-A., Kádár, A., Yuan, X., Kybartas, B., Barnes, T., Fine, E., Moore, J., Hausknecht, M., Asri, L. E., Adada, M., et al. Textworld: A learning environment for text-based games. In *Workshop on Computer Games*, pp. 41–75. Springer, 2018b.

Fulda, N., Ricks, D., Murdoch, B., and Wingate, D. What can you do with a rock? affordance extraction via word embeddings. *arXiv preprint arXiv:1703.03429*, 2017.

Goyal, P., Niekum, S., and Mooney, R. Pixl2r: Guiding reinforcement learning using natural language by mapping pixels to rewards. In *Conference on Robot Learning*, pp. 485–497. PMLR, 2021.

He, J., Chen, J., He, X., Gao, J., Li, L., Deng, L., and Ostendorf, M. Deep reinforcement learning with a natural language action space. *arXiv preprint arXiv:1511.04636*, 2015.

Huang, W., Abbeel, P., Pathak, D., and Mordatch, I. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents, 2022a. URL https://arxiv.org/abs/2201.07207.

Huang, W., Xia, F., Xiao, T., Chan, H., Liang, J., Florence, P., Zeng, A., Tompson, J., Mordatch, I., Chebotar, Y., Sermanet, P., Brown, N., Jackson, T., Luu, L., Levine, S., Hausman, K., and Ichter, B. Inner monologue: Embodied reasoning through planning with language models, 2022b. URL https://arxiv.org/abs/2207.05608.

Jang, E., Irpan, A., Khansari, M., Kappler, D., Ebert, F., Lynch, C., Levine, S., and Finn, C. Bc-z: Zero-shot task generalization with robotic imitation learning. In *Conference on Robot Learning*, pp. 991–1002. PMLR, 2022.

Jiang, Y., Gu, S. S., Murphy, K. P., and Finn, C. Language as an abstraction for hierarchical deep reinforcement learning. *Advances in Neural Information Processing Systems*, 32, 2019.

Kollar, T., Tellex, S., Roy, D., and Roy, N. Toward understanding natural language directions. In *2010 5th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pp. 259–266. IEEE, 2010.

Lin, B. Y., Huang, C., Liu, Q., Gu, W., Sommerer, S., and Ren, X. On grounded planning for embodied tasks with language models. *arXiv preprint arXiv:2209.00465*, 2022.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

MacMahon, M., Stankiewicz, B., and Kuipers, B. Walk the talk: Connecting language, knowledge, and action in route instructions. *Def*, 2(6):4, 2006.

Mei, H., Bansal, M., and Walter, M. R. Listen, attend, and walk: Neural mapping of navigational instructions to action sequences. In *Thirtieth AAAI Conference on Artificial Intelligence*, 2016.

Micheli, V. and Fleuret, F. Language models are few-shot butlers. *arXiv preprint arXiv:2104.07972*, 2021.

Min, S. Y., Chaplot, D. S., Ravikumar, P., Bisk, Y., and Salakhutdinov, R. Film: Following instructions in language with modular methods, 2021.

Misra, D., Langford, J., and Artzi, Y. Mapping instructions and visual observations to actions with reinforcement learning. *arXiv preprint arXiv:1704.08795*, 2017.

Nair, S., Mitchell, E., Chen, K., Savarese, S., Finn, C., et al. Learning language-conditioned robot behavior from offline data and crowd-sourced annotation. In *Conference on Robot Learning*, pp. 1303–1315. PMLR, 2022.

Oh, J., Singh, S., Lee, H., and Kohli, P. Zero-shot task generalization with multi-task deep reinforcement learning. In *International Conference on Machine Learning*, pp. 2661–2670. PMLR, 2017.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners. 2019.

See, A., Liu, P. J., and Manning, C. D. Get to the point: Summarization with pointer-generator networks. *arXiv preprint arXiv:1704.04368*, 2017.

Sharma, P., Torralba, A., and Andreas, J. Skill induction and planning with latent language. *arXiv preprint arXiv:2110.01517*, 2021.

Shridhar, M., Thomason, J., Gordon, D., Bisk, Y., Han, W., Mottaghi, R., Zettlemoyer, L., and Fox, D. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 10740–10749, 2020a.

Shridhar, M., Yuan, X., Côté, M.-A., Bisk, Y., Trischler, A., and Hausknecht, M. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768*, 2020b.

Shridhar, M., Manuelli, L., and Fox, D. Cliport: What and where pathways for robotic manipulation. In *Conference on Robot Learning*, pp. 894–906. PMLR, 2022.

Smith, S., Patwary, M., Norick, B., LeGresley, P., Rajbhandari, S., Casper, J., Liu, Z., Prabhumoye, S., Zerveas, G., Korthikanti, V., Zheng, E., Child, R., Aminabadi, R. Y., Bernauer, J., Song, X., Shoeybi, M., He, Y., Houston, M., Tiwary, S., and Catanzaro, B. Using deepspeed and megatron to train megatron-turing NLG 530b, A large-scale generative language model. *CoRR*, abs/2201.11990, 2022. URL https://arxiv.org/abs/2201.11990.

Song, C. H., Wu, J., Washington, C., Sadler, B. M., Chao, W.-L., and Su, Y. Llm-planner: Few-shot grounded planning for embodied agents with large language models. *arXiv preprint arXiv:2212.04088*, 2022.

Stepputtis, S., Campbell, J., Phielipp, M., Lee, S., Baral, C., and Ben Amor, H. Language-conditioned imitation learning for robot manipulation tasks. *Advances in Neural Information Processing Systems*, 33:13139–13150, 2020.

Tafjord, O. and Clark, P. General-purpose question-answering with macaw. *arXiv preprint arXiv:2109.02593*, 2021.

Tellex, S., Kollar, T., Dickerson, S., Walter, M., Banerjee, A., Teller, S., and Roy, N. Understanding natural language commands for robotic navigation and mobile manipulation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 25, pp. 1507–1514, 2011.

Yao, S., Rao, R., Hausknecht, M., and Narasimhan, K. Keep calm and explore: Language models for action generation in text-based games, 2020. URL https://arxiv.org/abs/2010.02903.

Zahavy, T., Haroush, M., Merlis, N., Mankowitz, D. J., and Mannor, S. Learn what not to learn: Action elimination with deep reinforcement learning. *Advances in neural information processing systems*, 31, 2018.